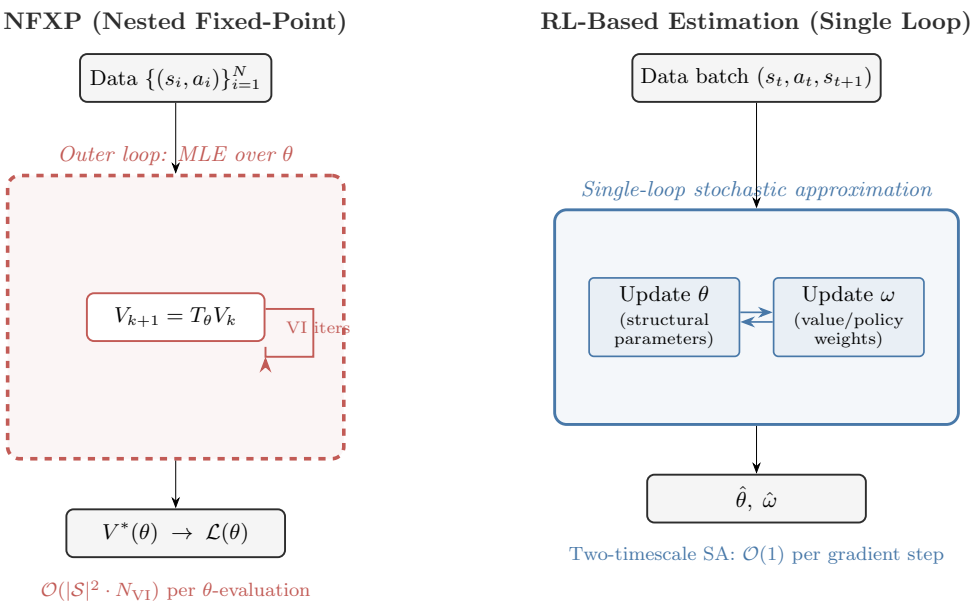

Figure 1: NFXP versus RL-based structural estimation. Top: the nested fixed-point algorithm evaluates the likelihood by solving the Bellman equation to convergence inside each optimizer step. Bottom: single-loop stochastic approximation updates structural parameters θ and value/policy weights ω simultaneously from data batches.

Several recent papers have used RL training loops, namely Q-learning, temporal-difference learning, policy gradient, and actor-critic methods, to solve structural economic models at scales where conventional dynamic programming fails.[1] Throughout this chapter I adopt a unified notation. An MDP is a tuple $(\mathcal{S}, \mathcal{A}, P, r, \gamma)$ where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, $P(s'|s,a)$ is the transition kernel, $r(s,a)$ is the per-period reward, and $\gamma \in [0,1)$ is the discount factor.[2] A policy $\pi : \mathcal{S} \to \Delta(\mathcal{A})$ maps states to distributions over actions. The value function under π is $V^\pi(s) = \mathbb{E}_\pi\left[\sum_{t=0}^\infty \gamma^t r(s_t, a_t) \mid s_0 = s\right]$, and the action-value function is $Q^\pi(s,a) = r(s,a) + \gamma \mathbb{E}_{s' \sim P(\cdot|s,a)}[V^\pi(s')]$. The optimal value function satisfies $V^*(s) = \max_{a \in \mathcal{A}} Q^*(s,a)$.

0.0.1 Adusumilli and Eckardt (2022): TD Learning for CCP Estimation

? adapt temporal-difference (TD) learning to estimate the recursive terms that arise in CCP-based estimation, entirely avoiding specification or estimation of transition densities.

The CCP approach requires computing two functions $h : \mathcal{A} \times \mathcal{S} \to \mathbb{R}^d$ and $g : \mathcal{A} \times \mathcal{S} \to$

[1] I exclude papers that use neural network function approximation without an RL training mechanism. Inverse reinforcement learning is treated in the sister survey (**?**).

[2] Several of the papers reviewed here use β for the discount factor, following economics convention. I translate all results to γ for consistency with the RL literature and the rest of this survey.

$\mathbb{R}$ that solve the recursive equations

$$h(a,s) = z(s,a) + \gamma\,\mathbb{E}[h(a',s') \mid a,s], \tag{1}$$

$$g(a,s) = e(a,s) + \gamma\,\mathbb{E}[g(a',s') \mid a,s], \tag{2}$$

where $e(a,s) = \gamma_{\mathrm{E}} - \ln P(a|s)$ under logit errors (γ_{E} denoting the Euler constant), and the expectation is over the next-period state-action pair (s',a') given the transition kernel $P(s'|s,a)$ and the observed policy $P(a|s)$. Both h and g have a value-function structure; they satisfy a Bellman-like recursion under the observed (data-generating) policy, not the optimal policy.

This observation motivates the approach. Standard TD learning estimates value functions under a fixed policy from trajectory data, which is exactly what is needed here. **?** propose two methods.

The core idea is to approximate h and g with linear combinations of basis functions, then use regression on observed state-action sequences to estimate the weights. This reduces the infinite-dimensional fixed-point problem to a finite-dimensional linear system.

The first is the linear semi-gradient method.[3] Approximate $h(a,s) \approx \phi(a,s)^\top w$ where $\phi : \mathcal{A} \times \mathcal{S} \to \mathbb{R}^p$ is a vector of basis functions (e.g., polynomials in state variables) and $w \in \mathbb{R}^p$ are the weights to be estimated. The TD(0) fixed-point equation is

$$\mathbb{E}\left[\phi(a,s)\left(\phi(a,s) - \gamma\phi(a',s')\right)^\top\right] w = \mathbb{E}\left[\phi(a,s)\, z(s,a)\right]. \tag{3}$$

The sample analog replaces population expectations with averages over the observed panel.

$$\hat{w} = \left(\frac{1}{n(T-1)}\sum_{i=1}^{n}\sum_{t=1}^{T-1}\phi_{it}\left(\phi_{it} - \gamma\phi_{i,t+1}\right)^\top\right)^{-1}\left(\frac{1}{n(T-1)}\sum_{i=1}^{n}\sum_{t=1}^{T-1}\phi_{it}\, z_{it}\right), \tag{4}$$

where $\phi_{it} = \phi(a_{it}, s_{it})$ and $z_{it} = z(s_{it}, a_{it})$. This requires inverting a $p \times p$ matrix, where p is the number of basis functions, making computation trivial in most settings. No transition density estimation is needed, since the method uses only observed sequences of current and next-period state-action pairs.

[3] The method is called "semi-gradient" because it computes the gradient of only the prediction $\phi(a,s)^\top w$ with respect to w, not the full TD error including the bootstrap target $\phi(a',s')^\top w$. This avoids differentiating through the target but sacrifices guaranteed convergence in some settings.

The second method is approximate value iteration (AVI). Rather than fixing a linear basis, AVI constructs a sequence of approximations $\hat{h}^{(k)}$ by iterating the Bellman-like operator and solving a nonparametric regression at each step. At iteration k, given the current approximation $\hat{h}^{(k-1)}$, one constructs pseudo-outcomes

$$Y_{it}^{(k)} = z_{it} + \gamma\, \hat{h}^{(k-1)}(a_{i,t+1}, s_{i,t+1}) \tag{5}$$

and then fits $\hat{h}^{(k)}$ by regressing $Y_{it}^{(k)}$ on (a_{it}, s_{it}) using any machine learning method, including LASSO, random forests, or neural networks. This is the first DDC estimator compatible with arbitrary ML prediction methods, enabling application to very high-dimensional state spaces.

With $\hat{h}$ and $\hat{g}$ in hand, structural parameters are recovered by pseudo-maximum likelihood estimation (PMLE). For continuous state spaces, **?** derive a locally robust correction to the PMLE criterion that accounts for the nonparametric first-stage estimation of value terms, restoring $\sqrt{n}$-convergence of $\hat{\theta}$.[4] The PMLE score is $m(a, s; \theta, h, g) = \partial_\theta \ln \pi(a, s; \theta, h, g)$, where π is the logit choice probability with continuation value $V(a, s) = h(a, s)^\top \theta + g(a, s)$. The naive estimator solves $\mathbb{E}_n[m(a, s; \theta, \hat{h}, \hat{g})] = 0$, but with continuous states this moment condition is not orthogonal to the first-stage estimates and converges slower than $\sqrt{n}$. The locally robust moment adds a debiasing correction:

$$\zeta = m(a, s; \theta, h, g) - \lambda(a, s; \theta) \left\{ z(s, a)^\top \theta + \gamma\, e(a', s') + \gamma\, V(a', s') - V(a, s) \right\}, \tag{6}$$

where $\lambda(a, s; \theta)$ solves a backward recursion that propagates the influence of estimation error through the dynamic structure.[5] The corrected estimator $\hat{\theta}_{LR}$ solves $\mathbb{E}_n[\zeta_n] = 0$ and is computationally no harder than the naive PMLE, since the correction is constant in θ.

Theorem 1 (**?**, Theorem 1)**.** *Under regularity conditions, the linear semi-gradient estimator $\hat{h}$ satisfies $\|\hat{h} - h\|_2 = O_P(n^{-1/2}(T-1)^{-1/2})$, where $\|\cdot\|_2$ denotes the $L^2(P)$ norm.*[6]

Theorem 2 (**?**, Theorem 5)**.** *Under regularity conditions on the ML method used in AVI,*

[4] An estimator has $\sqrt{n}$-convergence if its error shrinks at rate $n^{-1/2}$ where n is the sample size. This is the standard parametric rate; slower rates (e.g., $n^{-1/4}$) indicate efficiency loss from nonparametric first stages.

[5] The term in braces is the temporal-difference error of the continuation value V. The adjoint λ weights this TD error by its marginal impact on the PMLE score. See Online Appendix B.3 of **?** for the derivation.

[6] The $L^2(P)$ norm is $\|f\|_2 = (\int f(x)^2\, dP(x))^{1/2}$, measuring average squared deviation under the probability measure P. This is the natural norm for mean-squared-error analysis.

the locally robust PMLE estimator $\hat{\theta}_{LR}$ satisfies $\sqrt{n}(\hat{\theta}_{LR} - \theta^*) \xrightarrow{d} \mathcal{N}(0, \Sigma)$ for an explicit variance Σ, even when the state space is continuous.

Monte Carlo experiments on a dynamic firm entry model with seven structural parameters and five continuous state variables show that the TD-based estimators achieve a 4- to 11-fold reduction in mean squared error compared to CCP estimators using state-space discretization.

For dynamic discrete games, the method extends naturally. Standard CCP-based estimation of games requires integrating out other players' actions, which becomes intractable with many players or continuous states. TD learning avoids this entirely, since it works directly with the joint empirical distribution of states and their successors. The "integrating out" is done implicitly within sample expectations.

The limitation is the linear semi-gradient's dependence on the choice of basis functions, though cross-validation can guide this selection. AVI with flexible ML methods relaxes this constraint at the cost of slightly higher small-sample bias, which the locally robust correction addresses.

0.0.2 Hu and Yang (2025): Policy Gradient for DDC Estimation

? combine policy gradient methods with the Simulated Method of Moments (SMM) to estimate DDCs, with particular focus on models with unobserved state variables. Where the previous subsection replaced the inner fixed-point computation with TD learning, Hu and Yang replace it with policy gradient, sidestepping value function approximation entirely and working directly in the space of parametrized policies.

The outer loop is SMM.[7] Define a vector of data moments $\mathbf{M}_d$ computed from the observed panel. For candidate structural parameters θ and transition parameters ξ, simulate the model to produce simulated moments $\mathbf{M}_s(\theta, \xi)$. The estimator minimizes

$$(\hat{\theta}, \hat{\xi}) = \arg\min_{\theta,\xi} \left(\mathbf{M}_d - \mathbf{M}_s(\theta, \xi)\right)^\top \mathbf{W} \left(\mathbf{M}_d - \mathbf{M}_s(\theta, \xi)\right), \tag{7}$$

where $\mathbf{W}$ is a positive definite weight matrix. Computing $\mathbf{M}_s(\theta, \xi)$ requires solving for the optimal policy under (θ, ξ), which is the inner-loop problem.

[7] SMM (**?**) estimates structural parameters by matching moments from simulated data to moments from observed data, avoiding direct evaluation of the likelihood function.

The inner loop parametrizes the choice probability directly as a logistic function of state variables. For a binary choice $J_t \in \{0, 1\}$, the general form is $\Pr(J_t = 1 \mid \boldsymbol{X}_t; \boldsymbol{\gamma}(\theta)) = \text{logistic}(\boldsymbol{X}_t \boldsymbol{\gamma}(\theta))$, where $\boldsymbol{\gamma}(\theta)$ are policy parameters that depend on the structural parameters.[8] In their application to a Rust bus engine model with unobserved bus condition S_t^*, this takes the form

$$\Pr(J_t = 1 \mid X_t, S_t^*, t; \boldsymbol{\gamma}) = \frac{\exp(\gamma_0 + \gamma_1 t + \gamma_2 X_t + \gamma_3 S_t^*)}{1 + \exp(\gamma_0 + \gamma_1 t + \gamma_2 X_t + \gamma_3 S_t^*)}, \tag{8}$$

where t enters the index directly to capture time-varying replacement incentives. The policy parameters are updated by REINFORCE-style gradient ascent, applying the policy gradient theorem (**?**):

$$\nabla_\gamma V(\boldsymbol{\gamma}) = \mathbb{E}\left[\sum_{t=0}^{T} \nabla_\gamma \log \pi_\gamma(J_t \mid X_t, S_t^*, t)\, Q^{\pi_\gamma}(X_t, S_t^*, J_t)\right], \tag{9}$$

where Q^{π_γ} is the action-value function under the current policy, estimated by Monte Carlo returns from forward-simulated trajectories.

The main contribution is handling unobserved state variables. When state variables are only partially observed, the policy in (**??**) is parametrized as a function of both X_t and S_t^*, and the algorithm forward-simulates trajectories of both observed and unobserved variables.

Building on the nonparametric identification results of **?**, the outer-loop SMM targets moments from five consecutive periods of observed data, which suffice to separately identify the structural parameters θ and transition parameters ξ without requiring the econometrician to observe S_t^*. No discretization of continuous unobserved states is needed; the same algorithm handles both discrete and continuous unobserved heterogeneity.

For each candidate (θ, ξ) in the outer minimization (**??**), the inner loop runs policy gradient until convergence, producing optimal policy parameters $\boldsymbol{\gamma}^*(\theta, \xi)$. These are used to simulate data and compute $\mathbf{M}_s(\theta, \xi)$.

On an extended Rust bus engine model with a continuous unobserved bus condition following an AR(1) process, estimates of seven structural parameters are centered around their true values across 400 simulations (Table 1 in their paper). On a discrete-

[8] The linear index can be replaced by higher-order terms of $\boldsymbol{X}_t$ or deep neural networks; the method requires only that the gradient $\nabla_\gamma \log \pi_\gamma$ has a closed form.

unobservable variant of the same model, the method matches the precision of **?**'s two-step EM algorithm, with computation time depending on the number of inner-loop iterations: at 5,000 iterations, RL+indirect inference runs in 0.40 minutes versus 0.66 minutes for CCP estimation; at 20,000 iterations, it rises to 1.15 minutes (Table 2 in their paper). Each outer-loop evaluation requires a full policy gradient convergence in the inner loop, so the computational advantage diminishes as more inner-loop iterations are used for precision.

0.1 Dynamic Oligopoly and Strategic Interaction

Dynamic oligopoly models combine elements of game theory and dynamic programming. Firms choose actions (prices, quantities, investment levels) strategically, anticipating both their own future states and the strategies of competitors. The state space grows combinatorially in the number of firms, and the requirement to compute best responses against other firms' strategies adds further computational burden.

0.1.1 Asker, Fershtman, Jeon, and Pakes (2020): Q-Learning in Dynamic Procurement Auctions

? develop a computational framework for analyzing dynamic procurement auctions with serially correlated asymmetric information. Their approach builds on the Experience-Based Equilibrium (EBE) concept of **?**, which computes equilibria by simulating industry trajectories and updating strategies toward best responses. EBE evaluates values only on recurrently visited states rather than the full state space, making it feasible for large state spaces. While **?** use a stochastic approximation algorithm to update continuation values from simulated industry trajectories, **?** add explicit value-function updates via stochastic approximation.

The model is a repeated first-price sealed-bid auction with two firms. Each firm i maintains a private inventory state $\omega_{i,t}$ (stock of unharvested timber, in their application). The state evolves endogenously, as winning an auction increases inventory while harvesting depletes it. Each firm's private state is not observed by its competitor except at periodic revelation events.

The key computational innovation is the use of Q-learning to compute equilibrium strategies. Each firm i maintains a Q-function $Q_i : \mathcal{S}_i \times \mathcal{A}_i \to \mathbb{R}$, where $\mathcal{S}_i$ encodes firm

i's information set (its own inventory, beliefs about the competitor's inventory, public history) and $\mathcal{A}_i$ is its action set (participation decision and bid level). The Q-function satisfies

$$Q_i(s,a) = \mathbb{E}\left[r_i(s,a,a_{-i}) + \gamma \max_{a' \in \mathcal{A}_i} Q_i(s',a') \;\middle|\; s,a \right], \tag{10}$$

where $r_i(s,a,a_{-i})$ is firm i's per-period profit given the state, its own action a, and the competitor's action a_{-i}, and the expectation is taken over the competitor's strategy and the stochastic transitions.

Firms update Q-values using sample averaging.

$$Q_i(s,a) \leftarrow Q_i(s,a) + \frac{1}{h_k(s,a)} \left[r_i + \gamma \max_{a' \in \mathcal{A}_i} Q_i(s',a') - Q_i(s,a) \right], \tag{11}$$

where $h_k(s,a)$ is the number of times state-action pair (s,a) has been visited.[9] The equilibrium computation proceeds iteratively, with firms simultaneously updating their Q-functions based on simulated play against each other's current strategies. Strategies are derived from Q-values using an ε-greedy rule or Boltzmann exploration.[10]

? add a boundary consistency condition to the EBE concept that restricts behavior at the boundary of the recurrent state class, reducing the multiplicity of equilibria. Their numerical analysis reveals that information sharing between firms can, through increased precision of beliefs about competitor states, induce firms to spend more time in states where competition is less intense. The dynamic RL-computed equilibrium yields qualitatively different predictions from both static analysis and myopic ($\gamma = 0$) benchmarks. With dynamics, information sharing decreases average bids and increases average profits, while the myopic benchmark shows negligible effects.

The limitation of this approach is the tabular representation; the Q-function is stored as a lookup table over discretized states and actions, restricting applicability to models with moderate state-space dimension.

[9] This sample-averaging rule ($\alpha_k = 1/h_k$) is equivalent to maintaining the running mean of observed returns, as distinct from fixed-α Q-learning which gives exponentially decaying weight to older observations. The update is applied for all actions, including counterfactual actions not taken, using the observed state transition.

[10] ε-greedy selects the greedy action $argmax_a\, Q(s,a)$ with probability $1-\varepsilon$ and a uniformly random action otherwise. Boltzmann exploration selects action a with probability $\propto \exp(Q(s,a)/\tau)$ where $\tau > 0$ is a temperature parameter.

0.1.2 Hollenbeck (2019): TD Learning for Merger Analysis with Innovation

? uses RL to solve a dynamic oligopoly model with endogenous mergers, entry, exit, and quality investment. The model extends the Ericson-Pakes framework to study how horizontal mergers affect innovation incentives.

The industry state is $\Omega = (\omega_1, \ldots, \omega_n)$ where $\omega_i \in \{1, \ldots, \omega_{\max}\}$ is firm i's product quality. Firms produce differentiated goods and compete in prices (Bertrand competition with logit demand). In each period, firms simultaneously choose investment levels, entry/exit decisions, and potentially initiate merger negotiations.

Each firm i computes its continuation value $V_i(\Omega)$ from the industry state. Because the state space is the product of all firms' quality levels plus industry structure (number of active firms, recent mergers), exact dynamic programming is infeasible for industries with more than two or three firms. **?** instead uses temporal-difference learning to estimate values from simulated industry trajectories.

The value function update for firm i is[11]

$$V_i(\Omega) \leftarrow V_i(\Omega) + \alpha \left[\Pi_i(\Omega, \mathbf{a}) + \gamma V_i(\Omega') - V_i(\Omega)\right], \tag{12}$$

where $\Pi_i(\Omega, \mathbf{a})$ denotes firm i's per-period profit given industry state Ω and the joint action profile $\mathbf{a}$ (investment, entry/exit, merger decisions),[12] γ is the discount factor, and Ω' is the realized next-period state. The algorithm uses ε-decreasing exploration to prevent convergence to locally suboptimal equilibria.

The equilibrium computation follows the Pakes-McGuire iterative scheme.[13] The algorithm simulates long industry histories, updates each firm's value function via (**??**), re-derives best-response strategies from updated values, and repeats. The RL component replaces the exact Bellman iteration step that would be infeasible over the full state space, focusing computational effort on recurrently visited states.

The central finding is that horizontal mergers, while reducing static consumer surplus in the short run, create a strong incentive for entry and investment. Firms enter with

[11] This stochastic approximation update, introduced by **?** for dynamic oligopoly computation, is closely related to temporal-difference learning in the RL literature. The original algorithm uses visit-count averaging ($\alpha_k = 1/k$ where k counts visits to each state) rather than a fixed learning rate.

[12] I use Π_i for firm i's per-period profit to avoid confusion with policy π.

[13] The Pakes-McGuire algorithm (**?**) computes Markov perfect equilibria by iterating: (1) given current value functions, compute best-response strategies; (2) given strategies, update value functions via simulation. Convergence is not guaranteed but works well in practice.

negative static profits because the prospect of a lucrative buyout justifies the initial investment. The result is substantially higher long-run innovation and consumer welfare with mergers than without. This finding reverses the standard static antitrust prediction and can only emerge in a dynamic model where firms are forward-looking.

Two related papers merit brief mention. **?** develop structural estimation methods for strategic settings where agents use learning algorithms (specifically regret-minimizing rules) rather than playing a fixed equilibrium. They impose an "asymptotic no-regret" condition as a minimal rationality requirement and derive identification results for payoff parameters. Their contribution is conceptual, bridging the gap between RL-based behavioral models (where agents learn) and the econometric problem (where the researcher estimates structural parameters from data generated by learning agents). Their estimation method itself is standard econometrics rather than RL. **?** uses deep RL to study oligopolistic pricing in a New Keynesian framework, representing firms' pricing policies as neural networks $\pi_\phi(a|s)$. The method uncovers multiple equilibria ranging from competitive to collusive pricing.[14]

0.2 Auction Equilibria and Mechanism Design

Computing equilibrium bidding strategies in auctions is a long-standing challenge. Closed-form solutions exist only for narrow families of valuation distributions and auction formats, while numerical methods scale poorly with the number of bidders and items. Recent work has applied deep RL to learn equilibrium strategies and optimal mechanisms.

0.2.1 Brero, Eden, Gerstgrasser, Parkes, and Rheingans-Yoo (2021): RL for Sequential Price Mechanisms

? use RL to design optimal sequential price mechanisms (SPMs), a class of indirect auction mechanisms where agents are approached in sequence and offered menus of items at posted prices. SPMs generalize both serial dictatorship and posted-price mechanisms and essentially characterize all strongly obviously strategyproof (SOSP) mechanisms (**?**).[15]

[14] The most prominent example of algorithmic collusion is **?**, who showed that independent Q-learning agents in a repeated Bertrand pricing game learn to sustain supra-competitive prices and punish deviators without explicit communication. This result and its implications for competition policy are treated in the companion thesis chapter (**?**).

[15] A mechanism is strategyproof if truthful reporting is a dominant strategy. It is obviously strategyproof if this dominance is apparent even to boundedly rational agents; strongly obviously strategyproof (SOSP) adds that dominance holds at every information set (**?**).

The mechanism design problem is formulated as a partially observable Markov decision process (POMDP).[16] The state at round t includes the set of remaining items $\rho_t^{\text{items}} \subseteq [m]$, remaining agents $\rho_t^{\text{agents}} \subseteq [n]$, the partial allocation $\mathbf{x}_t$, and the agents' (unobserved) valuation functions. The action $a_t = (i_t, \{p_j^t\}_{j \in \rho_{t-1}^{\text{items}}})$ specifies which agent to visit next and what prices to offer. The observation is the agent's purchase decision, from which the mechanism can update beliefs about valuations. The reward is the objective function evaluated at the final allocation:

$$r = g(\mathbf{x}_T, \boldsymbol{\tau}_T; \mathbf{v}), \tag{13}$$

where g can be social welfare $\sum_i v_i(x_i)$, revenue $\sum_i \tau_i$, or max-min fairness $\min_i v_i(x_i)$.

A key theoretical result establishes when adaptive mechanisms outperform static ones.

Theorem 3 (**?**, Propositions 1–4, informal)**.** *Each feature of adaptive mechanisms is necessary for welfare optimality, even in simple settings. Personalized prices are needed with one item and two i.i.d. agents (Proposition 1); adaptive prices are needed with two identical items and three i.i.d. agents (Proposition 2); adaptive ordering is needed with six agents whose valuations are correlated (Proposition 3); both adaptive prices and ordering are needed with four agents whose valuations are independently but non-identically distributed (Proposition 4).*

The policy maps from a sufficient statistic of the observation history to actions. **?** show that this statistic can be represented compactly. The set of remaining items and agents suffices for independent valuations, while the full allocation matrix is needed for correlated valuations. They train the mechanism policy using Proximal Policy Optimization (PPO),[17] which handles the discrete action space (agent selection) and continuous action space (price setting) of the POMDP.

Experimental results show that the learned SPMs achieve near-optimal welfare across settings with up to 20 agents and 5 items (with similar results noted for up to 30 of each), significantly outperforming static pricing benchmarks. The improvement is largest when agent valuations are correlated, since adaptive prices allow the mechanism to infer

[16] In a POMDP, the agent cannot directly observe the full state. It maintains a belief distribution over possible states, updated via Bayes' rule as new observations arrive. This captures the mechanism designer's uncertainty about bidder valuations.

[17] Proximal Policy Optimization (**?**) is a policy gradient algorithm that constrains each update to a trust region, preventing large destabilizing policy changes. It is described in Chapter 2.

information about remaining agents from earlier purchases.

The limitation is that the POMDP formulation requires knowledge of the prior distribution over valuations, which in practice must be estimated from data. The method also does not scale easily to very large numbers of items due to the combinatorial action space.

0.2.2 Ravindranath, Feng, Wang, Zaheer, Mehta, and Parkes (2024): Fitted Policy Iteration for Combinatorial Auctions

? address revenue-maximizing mechanism design for combinatorial auctions with multiple items and strategic bidders. Their innovation is integrating differentiable auction structure into a fitted policy iteration framework, enabling analytical gradient computation where standard RL methods struggle with high variance.

The mechanism visits agents one at a time in sequence. Each agent i, upon being visited, selects a bundle of items from those still available, given posted prices. Valuations are drawn once from distributions V_i and remain fixed throughout the mechanism. Complementarities arise from the structure of the valuation function over bundles, not from dynamic evolution. The MDP state at step t is $s_t = (i_t, S_t)$, where i_t is the current bidder and S_t is the set of remaining items.

The mechanism's policy maps from state to a price vector over available items. The key technical contribution is making the auction clearing differentiable. In a standard auction, the allocation is an argmax over bids (non-differentiable), and payments depend discontinuously on the allocation. **?** replace the hard bundle selection with a softmax relaxation,[18] enabling analytical gradient computation through the mechanism. The actor loss is the negative expected revenue, and gradients flow directly through the softmax-relaxed allocation and payment rules. The paper explicitly avoids REINFORCE-style estimators, noting that analytical gradients overcome the sample inefficiency and high variance of score-function methods.

The method follows fitted policy iteration (**?**), alternating between evaluating the current policy (computing expected revenue) and improving it via gradient ascent on the actor loss. This differs from model-free RL approaches (PPO, SAC) that the paper uses as baselines.

[18] The softmax relaxation replaces the hard allocation $argmax_i\, b_i$ with a soft allocation $\exp(b_i/\tau)/\sum_j \exp(b_j/\tau)$, which is differentiable and approaches the hard allocation as $\tau \to 0$.

Experiments on settings with additive and combinatorial valuations show that the learned mechanisms achieve up to 13% higher revenue than item-wise Myerson optimal auctions, with the largest gains in combinatorial settings where bundle complementarities make item-wise pricing suboptimal. Standard RL baselines (PPO, SAC) are also outperformed, confirming the advantage of exploiting differentiable structure.[19]

0.3 Macroeconomic Models

Reinforcement learning and deep learning are increasingly used to solve high-dimensional macroeconomic models where grid-based dynamic programming is infeasible. Heterogeneous agent models, in which a distribution of agents with different wealth levels, productivities, or beliefs interact through markets, generate state spaces that grow with the number of agent types and asset positions. **?** demonstrate that deep neural networks can approximate policy and value functions in dynamic economic models, achieving accuracy comparable to established projection methods while scaling to problems with dozens of state variables. **?** solve a model with financial frictions and an endogenous wealth distribution using deep learning, obtaining global solutions to a problem where perturbation methods fail due to strong nonlinearities. **?** provide a systematic treatment of deep learning methods for high-dimensional dynamic programming problems in economics, covering both single-agent and equilibrium settings. **?** apply deep deterministic policy gradient (DDPG)[20] to a real business cycle model, demonstrating that model-free RL can recover near-optimal consumption and investment policies without deriving optimality conditions such as the Euler equation. This is a rapidly growing area; readers seeking a comprehensive methodological treatment are directed to the cited papers and the references therein.

0.4 Optimal Policy Design

? introduce the AI Economist, a two-level multi-agent reinforcement learning framework for automated tax policy design. In their environment, a population of AI worker agents learn to work, trade, and build in a spatial-economic simulation, while a government RL agent simultaneously learns tax brackets that optimize a social welfare objective. The

[19] DDPG was also tested but found to be unstable. DQN is not applicable to continuous price-setting.

[20] DDPG extends Q-learning to continuous action spaces by learning a deterministic policy network alongside a Q-function critic.

worker agents are trained with PPO to maximize individual post-tax utility, and the government agent is trained with PPO to maximize a weighted combination of equality (measured by the Gini index) and productivity (total output). The two-level structure produces a Stackelberg game between the government (leader) and the workers (followers), where the government must anticipate how tax policy changes affect worker behavior. The learned tax policies achieve equality-productivity tradeoffs that Pareto-dominate several analytical baselines, including the Saez tax formula. This framework extends the mechanism design perspective of the preceding subsection from auctions to fiscal policy, using multi-agent RL to jointly solve for optimal mechanisms and equilibrium responses.